

Editorial Pedagógico: Concurso de Contos (Simulação e Algoritmos Gulosos)

Por: Mundo do Código

Data: 23 de maio de 2026

Skills: Algoritmo Guloso, Simulação, Lógica Básica, Strings

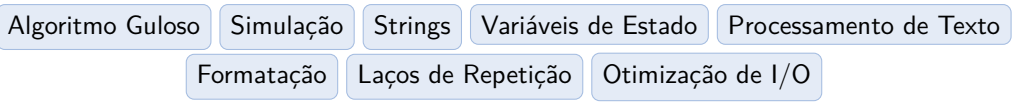
▷ Objetivo Pedagógico

- **Paradigma Guloso (Greedy):** Compreender e aplicar a escolha ótima local (alocar o máximo possível de palavras na linha atual) para atingir o ótimo global (minimizar o número de páginas).
- **Modelagem de Estado Discreto:** Desenvolver a capacidade de rastrear e atualizar múltiplas variáveis dependentes (caracteres, linhas, páginas) através de uma simulação passo-a-passo.
- **Tratamento de Casos de Borda:** Gerenciar adequadamente regras condicionais específicas, como o não-cômputo de espaços no início de linhas e transições exatas nos limites de capacidade.
- **Eficiência de I/O em Strings:** Analisar estratégias de leitura sob demanda (streaming) em contraposição ao carregamento completo de dados na memória.

1. Disciplinas Relacionadas

Disciplina	Tópicos Relevantes
Algoritmos e Estruturas de Dados	Paradigmas de projetos de algoritmos (Algoritmos Gulosos), Análise Assintótica.
Lógica de Programação	Estruturas de controle de fluxo, acumulação de estado, laços de repetição, simulação de processos.
Sistemas Operacionais	Gerenciamento de buffers, leitura de fluxos de entrada (streaming) vs. alocação contígua em memória.

Tabela 1: Mapeamento para currículos acadêmicos de Ciência da Computação.



2. Editorial Completo

O problema “Concurso de Contos” é um clássico de simulação baseada em restrições de formatação. O objetivo é determinar o número mínimo de páginas necessárias para imprimir um texto contínuo, dado um número predeterminado de caracteres por linha e linhas por página.

2.1. A Natureza do Problema: Formatação de Texto

Diferente de problemas avançados de diagramação (como o *Text Justification* ou o algoritmo do $\text{\TeX}$, que utilizam Programação Dinâmica para minimizar a “feiura” das quebras de linha), este problema não se importa com a estética do alinhamento direito. A única restrição aqui é o limite rígido superior de caracteres por linha, garantindo-se que cada palavra e seu respectivo espaço couberem no espaço restante da linha atual.

Isto simplifica a natureza do problema e permite uma abordagem direta, conhecida como **Algoritmo Guloso** (*Greedy Algorithm*).

2.2. O Paradigma Guloso

Um algoritmo guloso constrói uma solução global fazendo escolhas localmente ótimas a cada etapa. No contexto da diagramação de texto, a escolha gulosa se expressa da seguinte forma: “*Se uma palavra couber na linha atual, ela deve ser colocada nela. Caso contrário, e somente neste caso, move-se para a próxima linha.*”

Por que a escolha gulosa é ótima neste caso?

Poderia haver algum cenário em que mover uma palavra para a linha de baixo precocemente (mesmo havendo espaço na linha atual) resultaria em um número menor de páginas no final? A resposta matemática é não. Deixar espaço livre em uma linha sem necessidade apenas desloca todas as palavras seguintes para a frente, o que pode forçar novas quebras de linha mais adiante, potencialmente aumentando o número de páginas. Não existe penalidade para linhas com larguras variadas. Assim, maximizar a ocupação de caracteres em cada linha sempre minimizará o total de linhas usadas e, consequentemente, o número total de páginas.

2.3. Modelagem Matemática e Máquina de Estados

Para simular o processo, modelaremos o problema como uma máquina de estados finitos simples que atualiza suas variáveis a cada entrada de palavra.

Definimos o estado pelo triplete $(P_{atual}, L_{atual}, C_{atual})$, onde:

- P_{atual} : Número de páginas usadas até o momento (inicialmente 1).
- L_{atual} : Número de linhas ocupadas na página atual (inicialmente 1).
- C_{atual} : Quantidade de caracteres já escritos na linha atual (inicialmente 0).

Além disso, sejam C_{max} a quantidade máxima de caracteres por linha, e L_{max} a quantidade máxima de linhas por página. Considere uma sequência de palavras de comprimentos $w_1, w_2, \dots, w_N$.

Para o processamento da i -ésima palavra de comprimento w_i , calculamos a quantidade de espaço S_i que esta palavra exigirá na linha atual. Se for a primeira palavra a ser impressa na linha corrente ($C_{atual} = 0$), nenhum caractere de espaço é necessário antes dela. Se já houverem palavras na linha corrente ($C_{atual} > 0$), é obrigatório adicionar 1 caractere em branco como separador.

A regra pode ser expressa como:

$$S_i = \begin{cases} w_i, & \text{se } C_{atual} = 0 \\ w_i + 1, & \text{se } C_{atual} > 0 \end{cases}$$

A transição de estados se dá verificando se $C_{atual} + S_i \leq C_{max}$.

Caso 1: A palavra cabe na linha atual. O estado é atualizado adicionando a exigência ao acumulador de caracteres da linha atual:

$$C_{atual} \leftarrow C_{atual} + S_i$$

Caso 2: A palavra não cabe na linha atual. Um transbordamento ocorre. A linha deve ser finalizada.

$$L_{atual} \leftarrow L_{atual} + 1$$

$$C_{atual} \leftarrow w_i$$

Note que na nova linha, a palavra se torna a primeira, portanto, ocupará apenas w_i (sem espaço precedente).

Cascata de Eventos: Checagem de Página

Imediatamente após incrementar a linha atual (L_{atual}), surge uma ramificação lógica crítica: devemos verificar se o novo valor de L_{atual} excede o limite estrito da página. Se $L_{atual} > L_{max}$, então o estado transita para:

$$P_{atual} \leftarrow P_{atual} + 1$$

$$L_{atual} \leftarrow 1$$

A omissão dessa cascata é a principal fonte de falhas algorítmicas (Wrong Answer) neste desafio.

2.4. Estratégias de Leitura e Otimização de I/O

No enunciado, as palavras são descritas como texto integral, separadas por espaços em branco. Do ponto de vista da arquitetura de computadores, existem duas formas primárias de lidar com essa entrada:

1. **Materialização na Memória (*Buffered Read*):** Consiste em ler toda a sequência de caracteres de uma vez (por exemplo, usando `readline` até o fim), armazenar em uma string colossal na memória, aplicar uma função de particionamento (como `split`) gerando um grande arranjo de strings e, finalmente, iterar sobre o array calculando os comprimentos. Essa abordagem tem complexidade de espaço $\mathcal{O}(S_{total})$ e pode acionar instabilidades de coleta de lixo (*Garbage Collection*)

se a string for massiva.

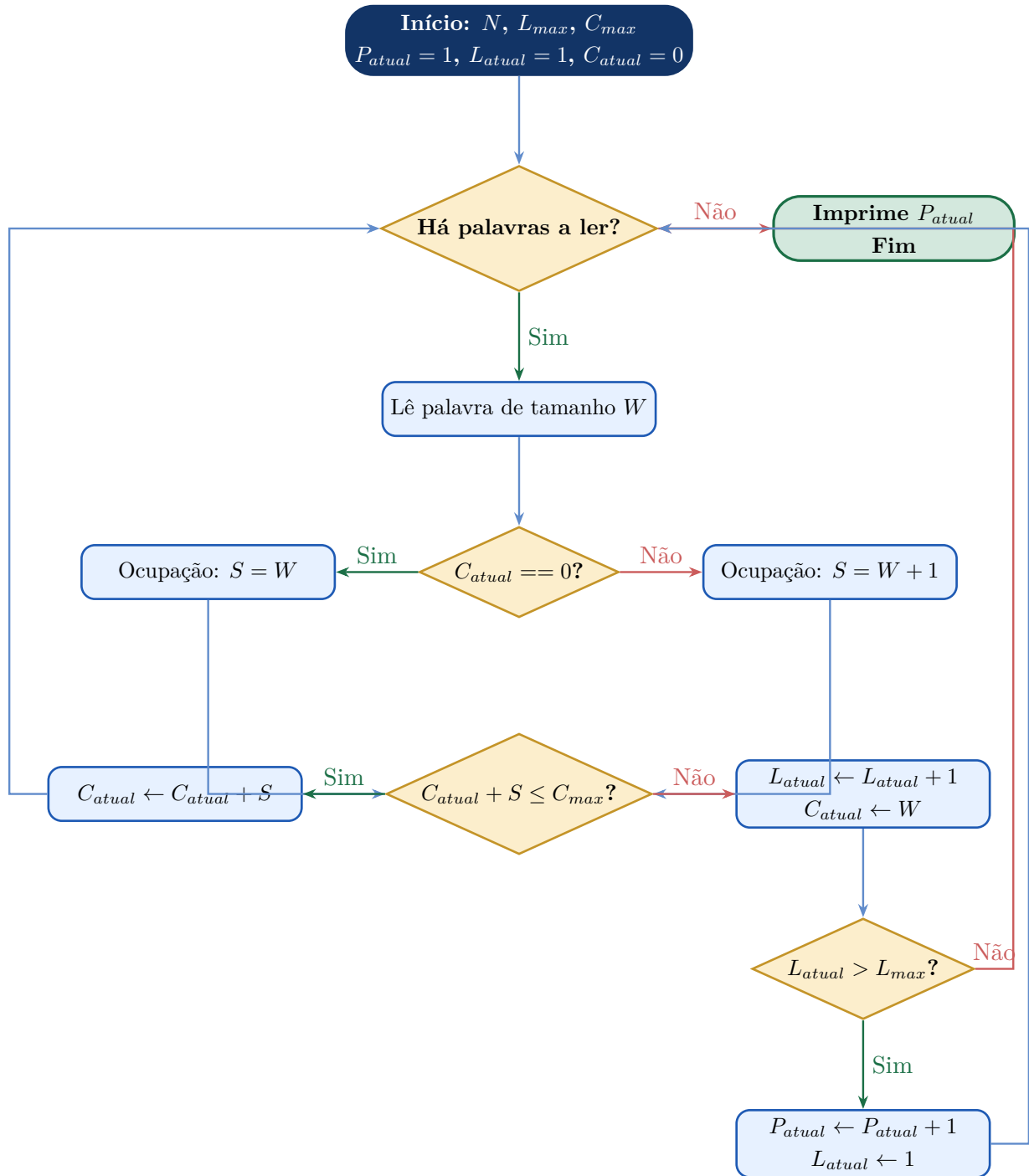
2. Processamento de Fluxo Contínuo (*Stream Processing*): Lê as palavras sob demanda, ignorando ativamente os separadores, tratando e descartando cada palavra em sequências mínimas da memória. O estado global (páginas, linhas, caracteres) é a única bagagem contínua.

Por que escolher o fluxo contínuo?

Em programação competitiva, problemas com alto volume de entrada sempre priorizam fluxos iterativos, pois minimizam gargalos de largura de banda na memória cache e operam com complexidade espacial próxima de $\mathcal{O}(1)$. Soluções maduras para este problema implementam um laço dependente de *tokens* padrão que consome uma palavra de cada vez, afere apenas seu atributo primordial (o tamanho) e passa para o próximo token.

3. Fluxograma da Solução

Abaixo, descrevemos visualmente a simulação adotada pelo Algoritmo Guloso usando a nossa máquina de estados.



4. Análise de Complexidade

A eficiência da solução varia sutilmente dependendo de como as strings são manipuladas. A estratégia gulosa exige unicamente uma varredura sequencial (da esquerda para a direita) pelo corpus da entrada. O processamento individual de uma palavra é constante dadas operações lógicas triviais. O que muda de fato é a alocação de memória e o tempo de construção de buffers temporários para *parsing* por debaixo dos panos.

Assumiremos que existam N palavras no texto total e que o comprimento total de caracteres em todo o texto (incluindo separadores) seja denotado por ΣS .

Etapa	Descrição	Tempo	Espaço
Abordagem Streaming	O analisador léxico extrai <i>tokens</i> on-the-fly a partir do <i>standard input stream</i> . Somente o tamanho de cada palavra lida é mantido.	$\mathcal{O}(N)$ ou $\mathcal{O}(\Sigma S)$	$\mathcal{O}(\max w_i)$
Abordagem Buffering	O programa aloca contiguamente toda a string e aplica separação léxica resultando num array de referências.	$\mathcal{O}(\Sigma S)$	$\mathcal{O}(\Sigma S)$
Lógica e Transição	Custo algorítmico do gerenciamento da máquina de estados por palavra lida (aritmética e branches).	$\mathcal{O}(1)$ por palavra	$\mathcal{O}(1)$

Tabela 2: Comparação das metodologias de leitura.

A simulação gulosa aliada ao processamento em fluxo é absolutamente a abordagem de ponta e performa em tempos consistentemente perto de 0.00s e consumo de memória da ordem de kilobytes nos ambientes de juiz online, operando perfeitamente bem aos ditames do limite espacial especificado no enunciado de 195 MB.

Tutorial Recomendado e Leituras Suplementares

Para quem deseja aprofundar os estudos em **Algoritmos Gulosos** na aplicação de problemas em grafos e processamento de dados, sugerimos a obra literária fundamental:

- *Introduction to Algorithms (Cormen, T. H. et al.)* — O Capítulo 16 descreve meticolosamente e formaliza as provas de matróides e teoremas subjacentes aos algoritmos Greedy.

Além do mais, compreender a distinção entre a versão simples que solucionamos neste editorial contra o famigerado **Text Justification Problem** ajudará incrivelmente na lapidação do seu raciocínio computacional:

- [Wikipedia: Line wrap and word wrap](#)